

Academy Backend/Frontend/QE Virtual MTY

Proyecto: 04-Unit Testing

Desarrollador: Sergio Servando Prado Lozano

Encargado: Miguel Ángel Rugerio Flores

Lugar: San Pedro de las Colonias Coahuila

Fecha: 29/08/2026

Concepto 4.1: Unit Testing con JUnit

Un test automatizado es una afirmacion sobre el codigo que una maquina puede comprobar por si sola sin intervencion humana. El testing unitario resuelve el miedo a modificar el sistema, ya que si algo se rompe por accidente, la prueba avisa de inmediato detallando la clase y la linea exacta del fallo. Sobre la version de la herramienta, el temario de la materia menciona JUnit 5 pero el codigo entregado usa JUnit 6.1.3. Esto es completamente valido porque los paquetes y anotaciones que se escriben siguen siendo los mismos, solo que ahora el motor interno exige Java 17 como minimo. Un record o un getter que solo guarda datos no necesita pruebas porque no decide nada. Se le hacen pruebas exclusivas al codigo que toma decisiones, al que calcula, al que rechaza y al que valida. Siempre hay que probar los limites, el limite menos uno, el limite mas uno, el vacio y el nulo. Todo en lo que el código pudiera corromperse.

Los proyectos como base son academiaMonterrey/Semana3/04-Unit-Testing en donde se encuentran los proyectos 01-junit-fundamentos, 02-junit-catalogo y 03-junit-datos. En estos tres primeros proyectos se aplico JUnit de forma progresiva.

- Fundamentos y ciclo de vida: En el proyecto 01, la clase CicloDeVidaTest java demuestra el uso de la anotacion BeforeEach para instanciar un escenario limpio antes de cada prueba individual. Ademas, AsercionesTest java agrupa multiples validaciones usando assertAll para que el test no se detenga en el primer fallo y reporte todo de un golpe.

- El catalogo y los rechazos: En el proyecto 02, la clase `ExcepcionesTest` java usa `assertThrows` para probar el camino de error, garantizando que el sistema lance `CupoLlenoException` al intentar inscribir a un alumno en un curso sin lugares. Tambien se agruparon las pruebas por estado usando `Nested` en `CursoNestedTest` java.
- Muchos datos de un solo golpe: En el proyecto 03, se ataco el problema de la duplicacion de codigo usando pruebas parametrizadas. En `CsvSourceTest` java se probaron decenas de fechas y limites, incluyendo el problema del 29 de febrero validando si el ano fue bisiesto o no separando el formato del significado real. Con solo treinta y dos metodos escritos a mano se lograron generar doscientos once tests reales.

Aquí a continuación se mostrarán los scripts ejecutados para ver que pasaba en cada uno de los tres proyectos antes de todos se ejecuta el comando `./mvnw test` para ver el antes y después de los scripts.

En el primer proyecto llamado 01-junit-fundamentos Se nos demuestra que el proyecto compilo correctamente descargando todas sus dependencias y ejecutando los veintidos tests originales con total exito.

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.academymty.academia.AsercionesTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.100 s -- in com.academymty.academia.AsercionesTest
[INFO] Running com.academymty.academia.BoletaTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.040 s -- in com.academymty.academia.BoletaTest
[INFO] Running com.academymty.academia.CicloDeVidaTest
== @BeforeAll -- una sola vez, antes de todo
-> @BeforeEach -- boleta nueva y limpia
<- @AfterEach -- llevamos 1 test(s)
-> @BeforeEach -- boleta nueva y limpia
<- @AfterEach -- llevamos 2 test(s)
-> @BeforeEach -- boleta nueva y limpia
<- @AfterEach -- llevamos 3 test(s)
== @AfterAll -- una sola vez, al final. Corrieron 3 tests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.027 s -- in com.academymty.academia.CicloDeVidaTest
[INFO] Running com.academymty.academia.DecimalesTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.academymty.academia.DecimalesTest
[INFO] Running com.academymty.academia.PrimeroTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in com.academymty.academia.PrimeroTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 6.790 s
[INFO] Finished at: 2026-08-29T12:07:23-06:00
[INFO] -----

```

Aquí después se ejecuto el script de ver-fallar.sh.

```

$ ./scripts/ver-fallar.sh

== 1. La suite tal y como esta en el repositorio ==
BUILD SUCCESS -- los 22 tests en verde.

== 2. Metemos el bug: 'promedio() >= 70' pasa a 'promedio() > 70' ==
74: return promedio() > MINIMA_APROBATORIA; // BUG INYECTADO
Un caracter. El alumno que saca exactamente 70 ahora reprueba.

== 3. La suite con el bug dentro ==
[ERROR] BoletaTest.ellimiteAlcanzadoPorPromedio:109 El promedio es 70 clavado: aprueba ==> expected: <true> but was: <false>
[ERROR] BoletaTest.ellimiteExacto:83 70 es aprobatorio: el reglamento dice '70 o mas' ==> expected: <true> but was: <false>

[ERROR] Tests run: 22, Failures: 2, Errors: 0, Skipped: 0

== 4. Lee bien ese numero ==
20 de 22 tests SIGUIERON EN VERDE con el bug dentro.

Los que probaban 90 aprueba y 50 reprueba no se enteraron de nada:
estaban lejos de la frontera, y el bug solo vive EN la frontera.

Los dos que cayeron son los que se pararon justo encima del 70.
Esos son los que valen. Esa es la leccion del proyecto 01:

    prueba el limite, el limite menos uno, el limite mas uno,
    el vacio y el nulo.

Cobertura alta no es lo mismo que estar protegido.

== 5. Devolviendo el codigo a su sitio ==
src/main/java/com/academymty/academia/Boleta.java identico al original (sha256 41d0305a2c19). Todo en orden.

```

Esta imagen demuestra que tener todo en verde no asegura la ausencia de errores. Al meter un error en la logica, la mayoría de las pruebas pasaron porque evaluaban valores seguros. Solo las pruebas diseñadas exactamente en el limite detectaron la falla.

Aquí va la ejecución del segundo proyecto 02-junit-catalogo antes del script el cual se ejecutaron todos los test correctamente y pasaron todos.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.academymty.academia.CondicionesTest
[WARNING] Tests run: 7, Failures: 0, Errors: 0, Skipped: 5, Time elapsed: 0.121 s -- in com.academymty.academia.CondicionesTest
[INFO] Running Un curso
[INFO] Running Cuando ya cerro inscripciones
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in Cuando ya cerro inscripciones
[INFO] Running Cuando ya esta lleno
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in Cuando ya esta lleno
[INFO] Running Cuando acaba de crearse
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in Cuando acaba de crearse
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in Un curso
[INFO] Running com.academymty.academia.EtiquetasTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.510 s -- in com.academymty.academia.EtiquetasTest
[INFO] Running com.academymty.academia.ExcepcionesTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s -- in com.academymty.academia.ExcepcionesTest
[INFO] Running com.academymty.academia.InyeccionTest
-> arrancando: TestInfo sabe quien es y como se llama
-> arrancando: Varias inyecciones a la vez
-> arrancando: @TempDir: escribir el acta en disco sin dejar basura
-> arrancando: TestReporter: dejar rastro en el reporte, no en la consola
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.200 s -- in com.academymty.academia.InyeccionTest
[INFO] Running El ciclo de vida de un curso, contado en orden
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in El ciclo de vida de un curso, contado en orden
[INFO] Running com.academymty.academia.TiempoTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.439 s -- in com.academymty.academia.TiempoTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 37, Failures: 0, Errors: 0, Skipped: 5
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.304 s
[INFO] Finished at: 2026-08-29T12:17:47-06:00
[INFO] -----
```

Después se ejecuto el script contar.sh a ver que es lo que sucedió.

```
$ ./scripts/filtrar.sh

== La misma suite, tres filtros ==
mvn test Tests run: 37, Failures: 0, Errors: 0, Skipped: 5 1.391s de tests
-DexcludedGroups=lento Tests run: 34, Failures: 0, Errors: 0, Skipped: 5 0.841s de tests
-Dgroups=rapido Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 0.110s de tests

== Lo que acabas de ver ==
Los @Tag("lento") son 3 de 37 tests -- un 8 % de la suite -- y se
llevan mas de la MITAD del tiempo de ejecucion (compara la primera
linea con la segunda). Cada uno genera un acta a 40 ms por alumno.

Y el filtro mas agresivo, -Dgroups=rapido, deja la suite en una
fraccion: de ~1.1 s a ~0.02 s. Sesenta veces menos.

Ese desequilibrio es la regla, no la excepcion: en cualquier
proyecto real un punado de tests que tocan disco, red o base de
datos acaba pesando mas que los cientos que no tocan nada.

De ahi el reparto practico:

antes de cada commit mvn test -Dgroups=rapido
antes de subir la rama mvn test -DexcludedGroups=lento
en el servidor de CI mvn test

AVISO SOBRE EL RELOJ: si cronometras estas tres corridas con un
reloj de pared vas a ver casi lo mismo (~2 s cada una), porque en
un proyecto de este tamano Maven tarda mas en arrancar que en
correr nada. Lo que se compara arriba es el tiempo de los tests.
Es lo unico que crece cuando tu suite crece.

Y fijate en el "Skipped: 4" de la corrida completa: son los tests
con assumptions y con @Disabled de CondicionesTest. NO fallaron,
ni siquiera corrieron. Un build verde con tests abortados dentro
sigue siendo un build que no proba esas rutas.

== La suite con nombre (@Suite) ==
mvn test -Dtest=SuiteDelCurso
Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
-> 15 tests de humo, sin los lentos. Definido en SuiteDelCurso.java.
```

Se demuestra la importancia de clasificar las pruebas usando etiquetas para separar aquellas que consumen mucho tiempo de ejecucion. Asi se pueden correr fracciones de la suite mejorando el desarrollo continuo. Tambien se evidencia el uso de asunciones, donde cuatro pruebas fueron abortadas al no cumplir con las condiciones requeridas por el entorno, demostrando que un reporte verde puede esconder secciones de codigo sin evaluar si no se presta atencion a los saltos.

Después va la ejecución del tercer proyecto 03-junit-datos antes del script el cual se ejecutaron todos los test (211) correctamente y pasaron todos.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.academymty.academia.CsvSourceTest
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.315 s -- in com.academymty.academia.CsvSourceTest
[INFO] Running com.academymty.academia.EnumSourceTest
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.080 s -- in com.academymty.academia.EnumSourceTest
[INFO] Running com.academymty.academia.MethodSourceTest
[INFO] Tests run: 64, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.163 s -- in com.academymty.academia.MethodSourceTest
[INFO] Running com.academymty.academia.RepeticionesTest
[INFO] Tests run: 43, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 s -- in com.academymty.academia.RepeticionesTest
[INFO] Running com.academymty.academia.TestFactoryTest
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s -- in com.academymty.academia.TestFactoryTest
[INFO] Running com.academymty.academia.ValueSourceTest
[INFO] Tests run: 35, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.049 s -- in com.academymty.academia.ValueSourceTest
[INFO] Results:
[INFO] Tests run: 211, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.871 s
[INFO] Finished at: 2026-08-29T12:26:20-06:00
[INFO] -----
```

Ahora se mostrará lo que sucede después de meterle el script de contar.sh.

```

$ ./scripts/contar.sh

== 1. Lo que hay ESCRITO en src/test/java ==
@Test          1
@ParameterizedTest 24
@RepeatedTest    3
@TestFactory     4
TOTAL           32 metodos

== 2. Lo que de verdad SE EJECUTA ==
Tests run: 211, Failures: 0, Errors: 0, Skipped: 0

== 3. Y clase por clase ==
CsvSourceTest      27 tests
EnumSourceTest     20 tests
MethodSourceTest   64 tests
RepeticionesTest   43 tests
TestFactoryTest    22 tests
ValueSourceTest    35 tests

== Lo que significa ==
Un metodo por cada FORMA de probar algo.
Un test por cada DATO con el que se prueba.

Escribir el caso 212 no cuesta un metodo nuevo: cuesta una linea en
una tabla. Y ahi esta el verdadero cambio -- no es que escribas menos
codigo, es que anadir un caso deja de dar pereza. Los casos limite se
prueban cuando probarlos es barato.

Si algo falla, el reporte no dice "curpsInvalidas fallo": dice cual de
los diez datos fallo, con su nombre. Por eso vale la pena el atributo
name = "..." en cada @ParameterizedTest.

```

Aquí se demuestra la enorme ventaja de usar pruebas con parametros para validar multiples variaciones de informacion. El script comprueba que escribiendo tan solo treinta y dos metodos a mano se logran generar doscientos once tests completamente independientes al momento de la ejecucion. Esto se logra metiendo diversas fuentes de datos permitiendo probar todos los escenarios invalidos y casos limite sin tener que duplicar lineas de codigo teniendo un mantenimiento bajo.

Tener pruebas que pasan no significa estar protegido. En el primer proyecto se incluyo el script `ver-fallar.sh` que introduce un bug cambiando un solo caracter de mayor o igual a mayor estricto en la regla de aprobacion. Al ejecutar el script, veinte de veintidos pruebas siguieron pasando en color verde porque evaluaban casos lejanos al limite. Solo cayeron las pruebas diseñadas para evaluar exactamente el limite. Esto demuestra que si los tests no caen al romper el codigo a proposito, realmente no estan probando nada.

Si no se usa JUnit o si solo pruebas casos felices, el equipo de desarrollo tendria que probar cada cambio manualmente. Esto hace que el proceso sea muy lento y propenso a regresiones silenciosas donde una funcionalidad vieja deja de operar sin que nadie se de cuenta hasta que revienta en produccion.

Concepto 4.2: Mockito Dobles

Un doble de prueba o mock es un objeto vacío fabricado en tiempo de ejecución al que se le pueden programar respuestas específicas y preguntarle que acciones recibió de parte del código principal. Resuelve el problema de aislar la clase que se está probando cuando sus dependencias son extremadamente lentas o tienen efectos irreversibles en el mundo exterior. Usando instrucciones clave como `when` para definir el comportamiento y `verify` para comprobar las interacciones, se logra probar la lógica pura sin depender del entorno.

Los proyectos como base son `academiaMonterrey/Semana3/04-Unit-Testing` en donde se encuentra el proyecto `04-mockito-dobles`.

Toda la aplicación práctica de este concepto se encuentra en el archivo `ServicioInscripcionTest.java`. Ahí se utilizó la anotación `InjectMocks` para construir el servicio principal e inyectarle los colaboradores falsos creados previamente con la anotación `Mock`. Para la programación de los dobles se usó `when` simulando búsquedas exitosas y para comprobar el comportamiento del servicio se usó `verify`, asegurando que el correo se manda solo cuando hay lugar y usando la directiva `never` para garantizar que jamás se manden confirmaciones a inscripciones rechazadas.

Aquí yéndonos hacer buenas prácticas, se decidió mockear estrictamente a RepositorioAlumnos porque simula consultas a una base de datos que agregan una latencia inmensa a las pruebas simulando así la lentitud de respuesta en ellas. De igual forma se mockeo al Notificador debido a que su función original manda correos reales a los usuarios, un efecto externo que sería un desastre ejecutar durante las pruebas automatizadas. Algo que sí jamás se debe mockear es la clase Curso.java, esta jamás se mockeo y se instancio de manera real usando new. Esto es porque Curso.java es una clase local, veloz y sobre todo, es quien contiene la regla de negocio que decide si hay cupo disponible o no.

Para demostrar el peligro de mockear incorrectamente el código que toma decisiones, el proyecto incluye el script la-mentira.sh que rompe la lógica del curso a propósito. Al correr el script, la clase SobreMockeoTest.java pasa la prueba en color verde a pesar del código roto porque cometió el error de mockear a la clase Curso.java. Al mockear la pieza que decide, el test aprueba su propio bug convirtiéndose en una mentira.

Aquí va la demostración o las pruebas de lo que se dice por escrito mostrando la ejecución del script y tests. Antes de todos se ejecuta el comando `./mvnw test` para ver el antes y después de los scripts.

Se ejecuto el comando y todo corrió perfectamente los test pasaron sin ninguna falla.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.academymty.academia.CaptorTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.873 s -- in com.academymty.academia.CaptorTest
[INFO] Running [lento] El servicio contra el repositorio REAL
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.567 s -- in [lento] El servicio contra el repositorio REAL
[INFO] Running com.academymty.academia.EspiaTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.085 s -- in com.academymty.academia.EspiaTest
[INFO] Running com.academymty.academia.EstrictezTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.academymty.academia.EstrictezTest
[INFO] Running com.academymty.academia.PrimerMockTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in com.academymty.academia.PrimerMockTest
[INFO] Running com.academymty.academia.RespuestasTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.038 s -- in com.academymty.academia.RespuestasTest
[INFO] Running El servicio de inscripcion
[INFO] Running cuando la base de datos falla
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s -- in cuando la base de datos falla
[INFO] Running cuando todo esta en orden
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in cuando todo esta en orden
[INFO] Running cuando el curso ya esta lleno
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in cuando el curso ya esta lleno
[INFO] Running cuando algo no existe
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in cuando algo no existe
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.049 s -- in El servicio de inscripcion
[INFO] Running MAL EJEMPLO: sobre-mockeo
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.044 s -- in MAL EJEMPLO: sobre-mockeo
[INFO] Running com.academymty.academia.VerificarTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.034 s -- in com.academymty.academia.VerificarTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 45, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.096 s
[INFO] Finished at: 2026-08-29T12:48:11-06:00
[INFO] -----
```

Ahora probaremos primero el script de por-que-mockear.sh a ver que es lo que sucede.

```
$ ./scripts/por-que-mockear.sh

== La misma prueba, con y sin dobles ==
con el repositorio REAL (300 ms) Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 2.331s
con dobles de Mockito Tests run: 40, Failures: 0, Errors: 0, Skipped: 0 1.166s

== Lee bien esos dos numeros ==
Cinco tests contra el repositorio real tardan MAS que los cuarenta
que usan dobles. Y la latencia simulada es modesta: 300 ms. Una
consulta real por red, con la base en otro servidor, no es mas rapida.

Multiplica: doscientos tests que consulten dos veces cada uno, a
300 ms, son dos minutos de espera. Y no es solo el reloj -- es que
a los dos minutos la gente deja de correr la suite antes de subir
codigo, y una suite que nadie corre no protege nada (guia 02, §05).

Esa es la primera razon para usar un doble: VELOCIDAD.
Pero no es la mas fuerte. Las otras dos son:

CONTROL con un mock provocas en una linea lo que con la pieza
real cuesta un escenario entero: la base caída, el curso
lleno, el timeout. Ver ServicioInscripcionTest.

SEGURIDAD el Notificador real manda correos DE VERDAD. Un test que
lo use manda correos a personas cada vez que alguien
ejecuta `mvn test`. Eso no se puede deshacer.

Ojo: esto NO dice que ConRepositorioRealTest sobre. Es el unico que
comprueba que el repositorio de verdad funciona. Lo que no puede
pasar es que TODA la suite sea así.
```

Queda demostrado la justificación principal para simular dependencias externas. Al comparar la ejecución contra un repositorio real frente a uno simulado, se observa como los doubles reducen el tiempo de espera drásticamente. Si no se aislaran las bases de datos lentas, la suite completa tardaría demasiados minutos, volviéndose fácil de operar para el desarrollo continuo.

Ahora que pasa si metemos el script de la-mentira.sh que sucede aquí todo estos son contextos diferentes.

```
$ ./scripts/la-mentira.sh

== 1. Las dos clases, con el código sano ==
ServicioInscripcionTest VERDE Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
SobreMockeoTest VERDE Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
Las dos en verde. Prueban lo mismo y las dos parecen razonables.

== 2. Rompemos la regla del cupo: 'lugaresDisponibles() == 0' -> '< 0' ==
40: return lugaresDisponibles() < 0; // BUG INYECTADO
Ahora un curso lleno dice que NO lo está. Se cuelan alumnos de más.

== 3. Las mismas dos clases, con el bug dentro ==
ServicioInscripcionTest ROJO Tests run: 6, Failures: 2, Errors: 0, Skipped: 0
[ERROR] ServicioInscripcionTest.noSeCuela Expected com.academymty.academia.CupoLlenoException to be thrown, but nothing was thrown.
[ERROR] ServicioInscripcionTest.rechazaSinConfirmar Expected com.academymty.academia.CupoLlenoException to be thrown, but nothing was thrown.
SobreMockeoTest VERDE Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

== 4. Ahí está la mentira ==
SobreMockeoTest sigue en VERDE con el bug dentro.

No es mala suerte ni un descuido: es la consecuencia exacta de haber
escrito @Mock sobre Curso. Al mockearlo, estaLleno() ya no ejecuta la
regla -- devuelve lo que el propio test escribía en el when(). El test
no comprueba "un curso lleno se rechaza": comprueba "si yo digo que
esta lleno, se rechaza". Aprueba su propia respuesta.

ServicioInscripcionTest lo caza porque construye el Curso con new. La
diferencia entre las dos clases es UNA LINEA.

La regla, entonces:

MOCKEA lo que DUELE lento, externo, no repetible, no determinista.
Los repositorios y el notificador: bien.

NO MOCKEES lo que DECIDE si tiene dentro la regla que estas probando,
va real. Curso: mal.

Y el corolario incomodo, que es el mismo de la guía 01: la cobertura
no distingue estos dos casos. Las dos clases "cubren" ServicioInscripcion
igual de bien. Solo una lo protege.

== 5. Devolviendo el código a su sitio ==
src/main/java/com/academymty/academia/Curso.java identico al original (sha256 a16c51bd60a5). Todo en orden.
```

Aquí se demuestra el enorme peligro de mockear objetos de dominio que contienen reglas de negocio. Al inyectar un error en la lógica de cupo, el test que usó mocks excesivos no se enteró del fallo y pasó en verde porque evaluó su propia respuesta falsa. Solo el test que usó instancias reales para el código que toma decisiones logró detectar la falla correctamente.

Ahora esto es importante ya que la necesidad de usar dobles de prueba, el proyecto incluye tambien el script por-que-mockear sh. Al ejecutarlo se comprueba la enorme diferencia de tiempo entre los dos enfoques. Correr solo cinco pruebas contra la implementacion real del repositorio toma muchisimo tiempo debido a la latencia simulada de trescientos milisegundos por consulta. Por otro lado, correr cuarenta pruebas usando los dobles fabricados toma apenas una fraccion de segundo. Esto demuestra que aislar las dependencias pesadas es obligatorio para mantener una suite ágil y eficiente por eso es muy importante usar los dobles.

Ya que, si no usas Mockito para simular dependencias lentas, tu suite completa tardaria minutos enteros en ejecutarse es mucho tiempo de espera, provocando que los programadores dejen de correrla antes de subir sus cambios. Además, el sistema enviaria correos electronicos reales a cada rato o intentaria hacer cobros verdaderos durante la integracion continua. Pero si abusas de la herramienta y mockeas de mas, destruyes el proposito de la prueba al dejar sin evaluar las reglas del negocio o lo que diseñes al momento de implementar las pruebas.